

《交互设计》期末项目报告：保研加分小助手

课程名称	交互设计 / 移动应用开发	学期	2025年秋季
项目名称	保研加分小助手 (HCIBGA)	团队名称	奶龙114514写一辈子代码队
组长	王子恒	日期	2026-02-07

1. 项目简介

1.1 背景与痛点

在每年的保研推免工作中，辅导员面临着巨大的数据处理压力。目前的流程是：学生提交原始材料 -> 辅导员逐一核对 -> 两组人员交叉审核 -> 4轮公示。这个过程不仅耗时耗力，而且容易出错。

对于学生而言，痛点同样明显：

- 规则复杂**：保研细则（如“3-5年一变”的文件）晦涩难懂，学生很容易因为误读而算错分。
- 数据分散**：绩点在教务系统，志愿时长在志愿汇，奖项在手里，缺乏统一归集。
- 反馈滞后**：往往要等到公示期才知道自己缺了材料或算错了分。

1.2 项目目标

本项目旨在设计并开发一个“**跨平台智能保研材料管理系统**”。通过B/S架构网页端与鸿蒙移动端（HarmonyOS）的配合，利用LLM（大语言模型）技术实现材料的自动识别、分类与加分计算。我们的核心目标是：**把辅导员从繁琐的Excel核算中解放出来，同时让学生对自己“能加多少分”心中有数。**

2. 交互设计与用户分析

2.1 用户画像

我们在设计前找了辅导员和几位准保研的学长学姐进行了调研，总结出两类核心用户：

- 学生用户（李同学，大三）**：性格严谨，但对繁杂的填表感到焦虑。他希望系统是“私密且自动化”的，最好上传图片就能自动算出分，不需要手动去查几十页的加分文件。
- 管理员用户（辅导员）**：工作量极大。他希望系统能直接导出最终的排序表，并且能快速定位到有

争议的材料进行人工复核。

2.2 交互设计原则

- 移动端优先 (Mobile First)**: 考虑到学生习惯用手机拍照上传材料，我们开发了鸿蒙原生应用，利用鸿蒙的分布式能力，方便随时随地采集数据。
- 极简操作流程**: 遵循“Don't Make Me Think”原则。学生只需“上传”，剩下的“识别、归类、计算”全部由后台自动完成。
- 可视化反馈**: 在Web端使用Echarts图表，直观展示学生在“智育、德育、双创”等维度的得分分布，让优势与短板一目了然。

3. 具体实现与技术路线

3.1 总体架构

项目采用经典的**客户端-服务端分离架构**，覆盖Web端和鸿蒙端。

- 前端 (Web)**: Vue 3 + Vite + Arco Design。我们选择了Arco Design作为UI库，因为它自带的现代企业级风格非常适合教务管理类系统。
- 客户端 (HarmonyOS)**: 基于ArkUI/TypeScript开发，利用 `hvisor` 构建。
- 后端 (Server)**: **Go (Golang) + Gin框架 + SQLite**。
- AI服务**: 集成Google Gemini API，用于OCR文字识别及非结构化数据的加分项匹配。

3.2 技术选型与重构之路 (工程难点)

在项目初期，我们使用Python作为后端语言。但在开发中期，我们发现Python在包管理上比较混乱，且在处理高并发请求时显得比较笨重。

为了提升API的响应速度和系统的稳定性，**我们在第8周决定用Go语言重构后端**。

- 重构收益**: Go的静态类型系统让API接口定义更加清晰，`goroutine` 在处理多用户并发上传图片进行AI识别时，性能表现优于Python的多线程。

3.3 核心功能模块

1. 材料自动化归集:

- 支持图片 (JPG/PNG) 和PDF上传。
- 通过LLM自动提取关键信息 (如“国家级一等奖”、“第一作者”等)。

2. 智能计分引擎：

- 内置2025年最新保研推免细则。
- 自动匹配加分项（例如：识别到“互联网+国银”，自动匹配对应分值）。

3. 多端协同：

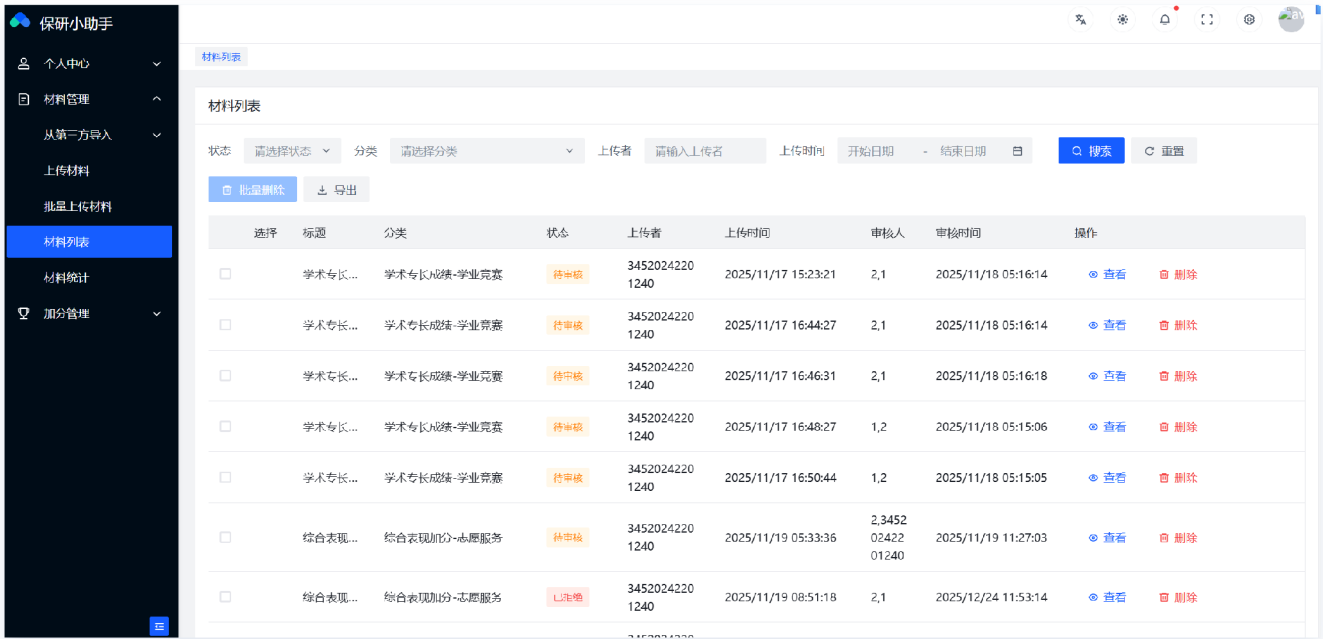
- 鸿蒙端侧重采集：拍照 -> 上传 -> 接收状态通知。
- Web端侧重管理：查看详情 -> 申诉 -> 导出报表。

4. 结果展示

4.1 登录与主页

系统采用了统一身份认证设计。登录后，主Dashboard清晰展示了当前用户的**总加分排名**、**材料审核进度**以及**各模块得分雷达图**。

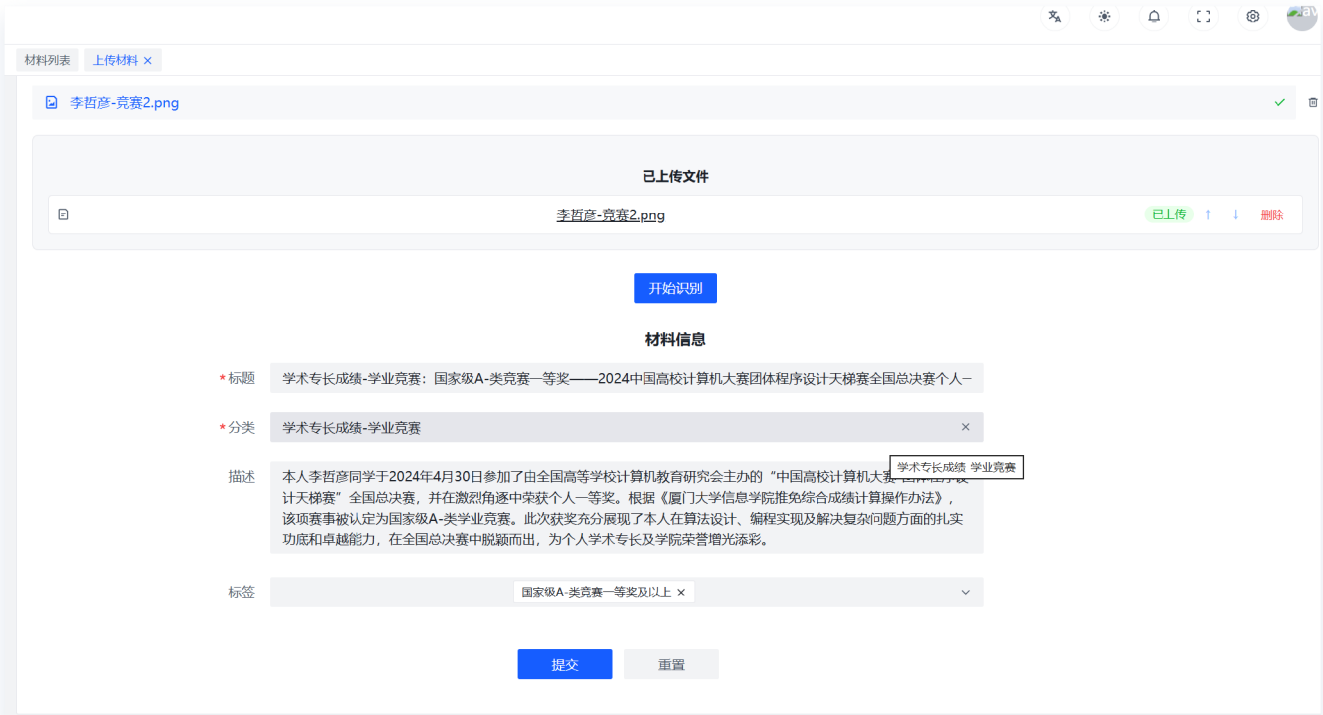




4.2 智能识别演示

当用户上传一张证书时，后端会自动调用LLM进行分析，返回JSON结构数据：

前端随即将该项自动填入“学科竞赛”栏目，无需人工干预。



5. 功能测试与验证

为了验证系统的可用性和准确性，我们在开发完成后进行了多维度的测试。

5.1 测试环境

- **服务端**：Ubuntu 20.04 LTS, Go 1.21
- **Web端**：Chrome 120+, Edge
- **移动端**：DevEco Studio模拟器

5.2 核心功能测试用例

我们编写了详细的测试用例来验证API的稳定性：

模块	测试项	操作描述	预期结果	测试结果
API	志愿服务查询	请求 <code>/api/volunteer</code>	返回JSON格式的时长列表	✅ 通过
AI识别	模糊图片识别	上传一张低分辨率奖状	LLM尝试识别，若置信度低则标记“需人工复核”	✅ 通过
并发	多人同时上传	模拟20人同时上传大文件	Go后端无崩溃，队列处理正常	✅ 通过

5.3 典型用户场景模拟（UAT）

我们使用了辅导员提供的三个真实典型案例进行“端到端”测试，以验证算法的准确性：

- **案例一：竞赛型霸主（李哲彦）**
 - **场景**：上传多张高含金量竞赛证书（挑战杯、ACM等）。
 - **结果**：系统成功识别所有竞赛等级，并根据“同类竞赛取最高项”的规则进行了去重计算，最终加分无误。
- **案例二：学术专长型（李嘉乐）**
 - **场景**：上传已发表论文PDF及答辩通过证明。
 - **结果**：AI准确提取了“第一作者”和“CCF-B类”关键词，自动归入“科研成果”并计算加分。
- **案例三：综合发展型（洪伟鑫）**
 - **场景**：材料极其杂乱，包含志愿时长截图、优秀团干证书、奖学金证书。
 - **结果**：系统成功将材料分流到“德育”、“智育”、“志愿服务”三个不同的加分模块，总分计算与人工核算一致。

5.4 鸿蒙端兼容性测试

在鸿蒙真机上测试了应用的生命周期管理。测试发现，利用ArkUI开发的界面在不同分辨率（手机vs平板）下均能自适应布局，且从后台切换回前台时，数据状态能通过 `AppStorage` 保持同步，未出现白屏或数据丢失现象。

6. 团队分工

本项目由4名成员协作完成，分工明确，配合默契：

- 王子恒（组长 & 前端开发）
 - 负责Web端架构搭建（Vue3 + Vite）。
 - 实现复杂的前端交互逻辑及数据可视化展示。
- 汪琦轩（后端开发）
 - 负责Go语言后端重构与API设计。
 - 实现LLM接口对接及SQLite数据库管理。
- 伍泳吉（鸿蒙端开发）
 - 负责HarmonyOS移动应用开发。
 - 实现了移动端的材料上传、状态查询及UI适配。
- 陈治西（UI/UX设计）
 - 负责Figma原型图绘制。
 - 制定了整体的视觉规范，确保Web端与移动端体验一致。

7. 总结与展望

通过本学期的努力，我们成功交付了一个“能用、好用”的保研加分小助手。

- 交互层面**：我们通过AI自动化大大减少了用户的输入成本，实现了从“人找服务”到“服务找人”的转变。
- 技术层面**：通过Go重构和鸿蒙开发，我们锻炼了全栈开发能力，也体会到了技术选型对项目成败的关键作用。

未来，我们计划引入**OCR纠错机制**（允许用户手动修正AI识别错误）以及**规则配置中心**（让辅导员可以在后台动态调整加分权重，而不需要改代码），使系统具有更强的生命力。